



SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

**Predicting Macroscopic Crowd Properties
Using the SWIM Algorithm**

Çağatay Gültekin





SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

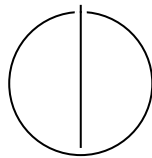
TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

**Predicting Macroscopic Crowd Properties
Using the SWIM Algorithm**

**Vorhersage makroskopischer
Crowd-Eigenschaften mit dem
SWIM-Algorithmus**

Author: Çağatay Gültekin
Examiner: Prof. Dr. Felix Dietrich
Supervisor: Ana Cukarska
Submission Date: 22.09.2025



I confirm that this master's thesis is my own work and I have documented all sources and material used.

Munich, 22.09.2025

Çağatay Gültekin

Acknowledgments

Abstract

Ensuring safe and efficient building evacuations is a central challenge in safety engineering and architectural design. Microscopic evacuation simulators such as `crowd:it` provide high-fidelity insights into crowd behavior, yet their computational cost makes them impractical for large-scale parameter studies or iterative design workflows. This thesis investigates surrogate modeling as a means to accelerate evacuation analysis by predicting macroscopic evacuation outcomes, specifically total evacuation times, at a fraction of the simulation cost.

We applied the SWIM algorithm (Sampling Where It Matters) to train neural surrogates on two complementary datasets. Scenario 1, a synthetic room-based setup, established a controlled baseline and confirmed that the method works in a simplified environment. Scenario 2, based on the regulation-derived Gd99 vertical-evacuation dataset, demonstrated applicability to real-world benchmarks and additionally served as the basis for evaluating sampling strategies. These included Quasi-Monte Carlo (QMC), Randomized QMC (RQMC), entropy-based, sparse-grid, and clustering approaches, applied to determine how few simulations are needed to train reliable surrogates.

The results show that SWIM-based surrogates can reproduce macroscopic evacuation outcomes with high accuracy while reducing simulation demand by more than an order of magnitude. Sampling strategy proved critical, with RQMC delivering the strongest generalization and achieving predictive accuracies above $R^2 = 0.95$ using only around a dozen simulations, while other methods required substantially larger sample sets for comparable performance.

Overall, this thesis confirms that surrogate modeling with SWIM is a promising tool for both research and applied safety practice. By integrating such approaches into existing simulation platforms, evacuation analysis can be performed rapidly, supporting both regulatory compliance and innovative building design.

Contents

Acknowledgments	iv
Abstract	v
1 Introduction	1
2 State of the Art	4
2.1 Background and Fundamentals	4
2.1.1 Human Behavior in Evacuations	4
2.1.2 Building Codes and Regulatory Context	7
2.2 Related Work	7
2.2.1 Evacuation Simulation Frameworks	7
2.2.2 Surrogate Modeling and the SWIM Algorithm	7
2.2.3 Sampling Strategies in Surrogate Modeling	8
3 AI-based Prediction of Evacuation Times Using Simulation Data	10
3.1 Scenario 1: Room-Based Prediction	10
3.1.1 Data Preparation	10
3.1.2 Model Design & Training	13
3.1.3 Results	16
3.1.4 Discussion	20
3.2 Scenario 2: Vertical Evacuation Prediction	21
3.2.1 Data Preparation	21
3.2.2 Model Design & Training	23
3.2.3 Results	24
3.2.4 Discussion	34
3.2.5 Scenario 2: Sampling Strategies	34
4 Conclusion and Future Work	39
Abbreviations	41
List of Figures	42

Contents

List of Tables	45
Bibliography	46

1 Introduction

Ensuring safe and efficient building evacuations is a longstanding challenge in engineering, architecture, and safety science. Traditional prescriptive rules, such as minimum stair widths or exit counts (Bareth, 2017; Kneidl & Könnecke, 2020), provide baseline guidance but cannot capture the full variability of human behavior or architectural design. Microscopic simulation tools, such as `crowd:it`, have therefore become essential for evaluating evacuation performance in complex layouts. However, detailed simulations are computationally expensive, especially when one must systematically vary scenario and population parameters within a given design (e.g., occupant load, pre-movement distributions, routing priorities). This creates a bottleneck for both research and practice. As models grow more realistic and detailed, their use in iterative design workflows becomes increasingly impractical.

Although simulation-based evacuation analysis is widely established, its computational cost remains a persistent barrier for large-scale or iterative studies. Surrogate modeling has been proposed in related engineering domains as a way to approximate costly simulations with lightweight predictors, yet its adoption in evacuation science has been limited. The recently introduced SWIM algorithm provides a fast, data-driven alternative to iterative training for neural networks, showing competitive accuracy across domains such as classification benchmarks, scientific operator learning, and transfer learning (Bolager, Burak, Datar, et al., 2023). While its potential for building surrogates of simulation-based evacuation models is evident, a systematic evaluation in the context of macroscopic crowd properties is still missing.

Equally critical is the question of *which* simulations should be selected for training. While sampling strategies such as Quasi-Monte Carlo, entropy maximization, or sparse grids are well established in numerical analysis, their comparative performance for evacuation prediction has not been benchmarked. To date, no study has combined surrogate modeling with a structured assessment of sampling methods on both synthetic and regulation-based evacuation datasets.

The aim of this thesis is to investigate whether surrogate models can serve as accurate and efficient predictors of evacuation times while substantially reducing simulation costs. The work pursues three interrelated objectives:

1. **Evaluate surrogate modeling with SWIM:** Assess the ability of SWIM-based neural surrogates to reproduce macroscopic evacuation outcomes with high

predictive accuracy.

2. **Benchmark sampling strategies:** In the context of Scenario 2, compare a range of sampling methods, including Quasi-Monte Carlo, Randomized QMC, entropy-based, sparse-grid, and clustering approaches, to identify which best balance accuracy with simulation efficiency.
3. **Validate across complementary scenarios:** Demonstrate the approach on two distinct datasets:
 - Scenario 1: A synthetic, room-based setup highlighting controlled parameter effects.
 - Scenario 2: A vertical-evacuation dataset derived from the Gd99 safety regulation (Kneidl & Könnecke, 2020) and simulated in the agent-based software `crowd:it` (accu:rate GmbH, 2023), providing a benchmark grounded in real-world design standards.

Together, these objectives address the central question of whether reliable evacuation predictions can be obtained with an order of magnitude fewer simulations than traditional workflows.

This thesis makes the following contributions:

- **Synthetic dataset for controlled evaluation:** We constructed Scenario 1, a synthetic, room-based setup generated via a Python script automating `crowd:it` simulations, highlighting the effects of randomized input parameters in a controlled setup.
- **Surrogate modeling with SWIM across evacuation scenarios:** We demonstrate that the general-purpose SWIM training paradigm (Bolager, Burak, Datar, et al., 2023) can be successfully adapted to predict macroscopic evacuation outcomes from microscopic simulations. Two complementary parameterizations were validated: (i) Scenario 1 with room-based parameters (N_1, N_2, N_3) , and (ii) Scenario 2 with regulation-based vertical-evacuation parameters (E, T, P) .
- **Systematic evaluation of surrogate accuracy:** We provide a reproducible pipeline for training and validating SWIM-based surrogates, showing that predictive accuracies of up to $R^2 = 0.89$ in the synthetic scenario and $R^2 = 0.99$ in a regulation-based scenario against ground-truth simulations.
- **Budgeted sampling study in Scenario 2:** We conducted a comparative study of established sampling strategies including Quasi-Monte Carlo (Sobol', 1967), Randomized QMC (Owen, 1998), entropy-based sampling (Cover & Thomas,

1991; Shannon, 1948), sparse grids (Stoyanov, 2017), and clustering-based methods (Arthur & Vassilvitskii, 2006; Lloyd, 1982) under fixed simulation budgets, highlighting their relative strengths and limitations. We achieved $R^2 = 0.95$ with the RQMC method using only a dozen simulations, demonstrating substantial gains in simulation efficiency.

- **Empirical insights on accuracy-cost trade-offs:** We quantified how surrogate accuracy scales with simulation budgets, showing that randomized QMC (Owen-scrambled Sobol' (Owen, 1998)) consistently achieves the most robust performance at low sample counts.
- **Practical workflow for design integration:** We demonstrate how surrogate-based approaches can reduce simulation costs in iterative evacuation analysis, with Scenario 2 linking directly to regulation-based benchmarks (Kneidl & Könnecke, 2020) and to simulation practice in `crowd:it` (accu:rate GmbH, 2023).

The remainder of this thesis is organized as follows.

Chapter 2: Reviews the state of the art in evacuation modeling, building codes, simulation frameworks, surrogate modeling, and sampling strategies.

Chapter 3: Presents Scenario 1, including data preparation, surrogate model training, results, and discussion. It then extends the study to Scenario 2, first validating surrogate modeling on the Gd99-derived vertical evacuation dataset and subsequently evaluating sampling strategies on this benchmark, with analysis and discussion integrated.

Chapter 4: Concludes the thesis and outlines directions for future work.

2 State of the Art

2.1 Background and Fundamentals

2.1.1 Human Behavior in Evacuations

Understanding crowd dynamics requires accounting for both individual decision-making and collective patterns. Microscopic behaviors such as route choice, group cohesion, and especially pre-evacuation delay are critical drivers of evacuation efficiency (Kuligowski, 2008). Empirical and modeling work shows that simple local rules of perception and movement can generate large-scale phenomena such as congestion and turbulence, underscoring the role of psychological and social factors during evacuations (Moussaïd, Helbing, & Theraulaz, 2011).

Pre-evacuation time is commonly defined as the period from cue recognition or notification until the occupant begins to move toward safety. Assumptions that everyone moves immediately after alarm can bias results in either unsafe or overly conservative directions (Kuligowski, 2008). This staged decision process is commonly conceptualized as a sequence of perception, interpretation, decision-making, and actions (see Fig. 2.1).

Modeling approaches are often divided into *macroscopic* and *microscopic* categories. Macroscopic models treat pedestrian motion as continuous flows (e.g., density-velocity relations), which are helpful for capacity checks but limited in behavioral realism. Microscopic and agent-based models simulate individuals and their interactions, capturing emergent effects such as lane formation and stop-and-go waves (Bandini, Manzoni, & Vizzari, 2009; Schadschneider, Klingsch, Klüpfel, et al., 2011; Zhou, Chen, Cai, et al., 2010). In agent-based terms, *agents* are typically characterized by autonomy, social ability, reactivity, and pro-activeness (Bandini, Manzoni, & Vizzari, 2009), as illustrated schematically in Fig. 2.2. Modern simulators such as *crowd:it* (accu:rate GmbH, 2023) are grounded in these microscopic principles, implementing the Optimal Steps Model, a locomotion approach grounded in empirical pedestrian studies, to reproduce observed phenomena such as bottleneck flows, congestion waves, and lane formation.

Empirical studies highlight that geometry strongly governs flows at bottlenecks and on stairs, which are key determinants of total evacuation time. Bottleneck experiments report pulsing and temporary blockages at narrow widths and show that simply increasing the width does not always increase flow. Similarly, stair motion

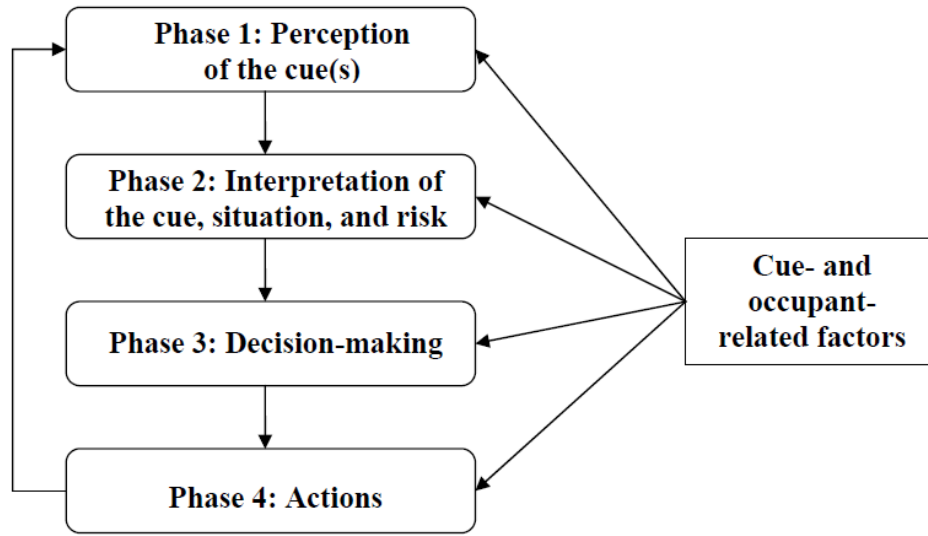


Figure 2.1: Conceptual model of occupant behavior in building fires: perception, interpretation, decision-making, and actions (Kuligowski, 2008).

exhibits reduced speeds and depends on riser/tread geometry and direction of travel (Schadschneider, Klingsch, Klüpfel, et al., 2011). These observations align with classic *escape panic* features such as arching and clogging at exits and jam formation (Helbing, Farkas, & Vicsek, 2000). In the context of this thesis, these geometric constraints directly motivate the regulation-oriented parameters in Scenario 2, where stair characteristics (T) play a central role. Figure 2.3 visualizes these effects, showing how compression and *crowd pressure* concentrate in hazardous zones near narrow exits.

Experimental insights into bottlenecks and stair flows have therefore become key validation targets for modern evacuation simulators. These tools adopt microscopic principles to reproduce phenomena such as congestion waves, lane formation, and stop-and-go dynamics, while balancing behavioral realism against computational efficiency. This trade-off is especially relevant for iterative design studies (Zhou, Chen, Cai, et al., 2010).

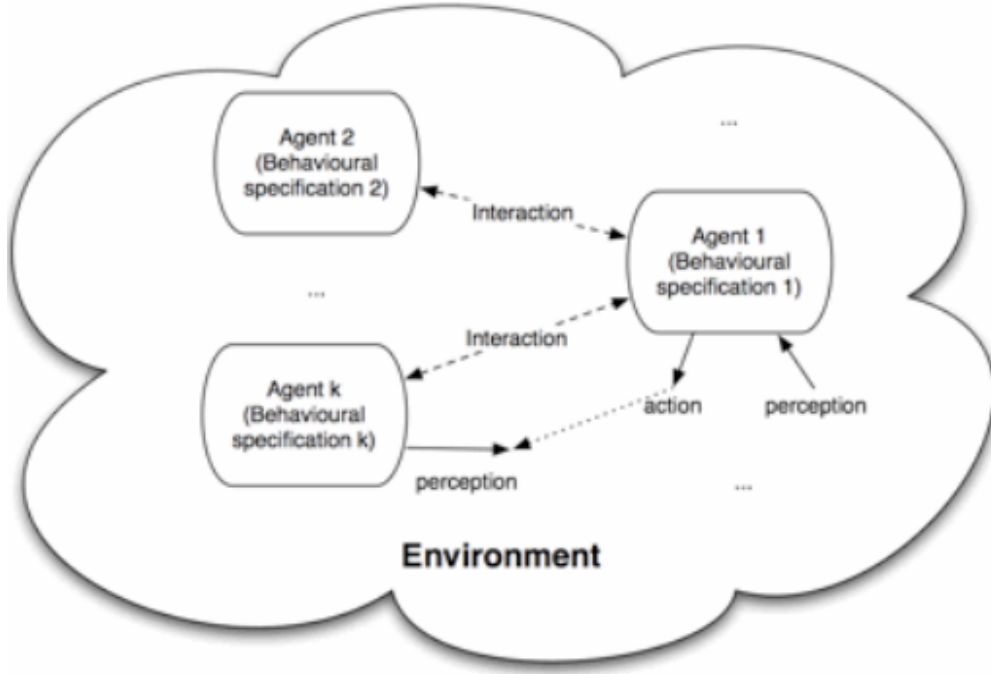


Figure 2.2: Schematic of an agent-based framework, where autonomous entities interact with each other and their environment (Bandini, Manzoni, & Vizzari, 2009).

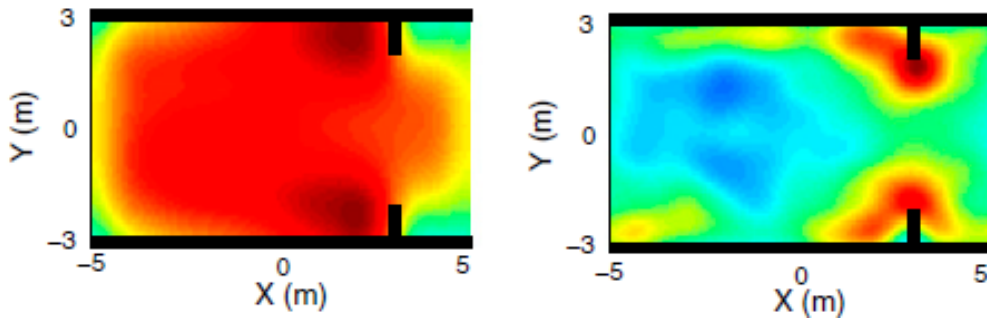


Figure 2.3: Local compression (left) and *crowd pressure* (right) in front of a bottleneck, illustrating congested and potentially hazardous zones near exits (Moussaïd, Helbing, & Theraulaz, 2011).

2.1.2 Building Codes and Regulatory Context

Evacuation safety in Germany is primarily governed by prescriptive building regulations. The *Musterbauordnung* (MBO) and the *Arbeitsstättenrichtlinie* (ASR A2.3) define baseline requirements such as minimum stair widths, door capacities, and evacuation procedures. Bareth (2017) consolidates these provisions and discusses their application in practical building design.

Although such prescriptive rules establish clear baselines, they cannot account for the variability of human behavior or the diversity of architectural layouts. This limitation has led to the integration of simulation into regulatory practice, where simulation studies complement and extend traditional code requirements.

The Gd99 expert report (Kneidl & Könnecke, 2020) exemplifies this approach. It introduced a series of vertical evacuation case studies, parameterized by the number of floors, stair widths, and occupant loads, explicitly designed as reference scenarios for validation and comparison. These scenarios were subsequently implemented in *crowd:it* (accu:rate GmbH, 2023), forming the dataset that underpins Scenario 2 in this thesis. Accordingly, the experiments presented here are directly rooted in regulatory practice while simultaneously serving as empirical benchmarks for simulation-based evaluation.

2.2 Related Work

2.2.1 Evacuation Simulation Frameworks

Several simulation frameworks have been developed to analyze evacuation scenarios. Tools such as *crowd:it* (accu:rate GmbH) and ASERI (BAuA) have been validated against the standardized RiMEA test cases (RiMEA e.V., 2016) and benchmarked against international reviews by NIST (Kuligowski & Peacock, 2005), demonstrating their ability to replicate both laboratory experiments and real evacuation drills. These simulators provide the flexibility needed to model complex geometries that go beyond the assumptions of prescriptive regulations. Contemporary platforms also incorporate pre-evacuation behavior, group effects, and route-choice logic informed by experimental and field data (Kuligowski, 2008).

2.2.2 Surrogate Modeling and the SWIM Algorithm

While microscopic evacuation simulations can reproduce complex dynamics with high fidelity, their computational cost becomes prohibitive when large parameter studies are required. To address this challenge, surrogate modeling approaches have emerged.

Bolager, Burak, Datar, et al. (2023) introduced the *Sampling Where It Matters* (SWIM) algorithm, a data-driven method for constructing efficient predictors without gradient descent. SWIM selects pairs of input points and sets each neuron’s weight and bias so that its activation attains prescribed values at those two points. Pairs are sampled with a data-dependent probability that favors large output changes over small input distances, yielding accurate surrogates at a fraction of the cost of iterative training (Fig. 2.4).

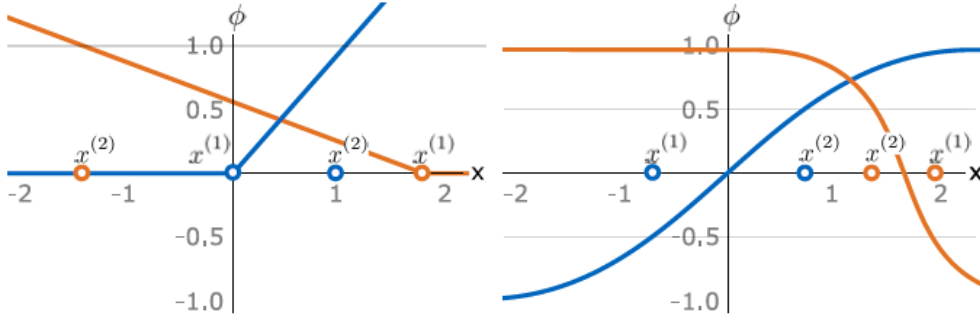


Figure 2.4: Point pairs $x^{(1)}, x^{(2)}$ for ReLU (left) and tanh (right). For example, SWIM enforces $\phi(x^{(1)}) = 0$, $\phi(x^{(2)}) = 1$ for ReLU, and $\phi(x^{(1)}) = \frac{1}{2}$, $\phi(x^{(2)}) = -\frac{1}{2}$ for tanh (with midpoint = 0). Weights and biases are thus fixed directly from data instead of gradient descent, illustrating the SWIM construction (reprinted from Bolager, Burak, Datar, et al. (2023)).

In this thesis, we apply SWIM to evacuation dynamics, where it serves as the methodological foundation for Scenario 1 and Scenario 2 experiments.

Beyond its original formulation, SWIM-based sampling has also been applied to accelerate the training of Hamiltonian graph networks in physics-informed settings, demonstrating its versatility as a surrogate modeling strategy (Rahma, Datar, Cukarska, & Dietrich, 2025).

2.2.3 Sampling Strategies in Surrogate Modeling

An additional line of research focuses on sampling strategies that minimize the number of required simulations while preserving accuracy. Classic information-theoretic ideas (Cover & Thomas, 1991; Shannon, 1948), clustering (Arthur & Vassilvitskii, 2006; Lloyd, 1982), low-discrepancy sequences (Sobol’, 1967), randomized scrambling (Owen, 1998), and sparse grids (Stoyanov, 2017) have all been adapted to surrogate modeling. While Quasi-Monte Carlo (QMC) and entropy-based strategies provide theoretical guarantees in continuous domains, their practical benefit depends on how effectively these designs

can be mapped to the discrete (E, T, P) parameter space characteristic of evacuation scenarios.

3 AI-based Prediction of Evacuation Times Using Simulation Data

3.1 Scenario 1: Room-Based Prediction

3.1.1 Data Preparation

Scenario Geometry and Task

Scenario 1 uses a controlled, simplified layout with three rooms (origins) leading to a single target area (see Figure 3.1). Each simulation is parameterized by the initial occupancies of the rooms, N_1 , N_2 , and N_3 . The prediction target is the total evacuation time.

The maximum occupancy per room was capped at 30 agents. This follows the area-density limits of approximately 0.5 persons/m², recommended for classroom or office-type spaces (Bareth, 2017), applied to the 60 m² room geometry. The door to the target area was set to a width of 1.20 m, corresponding to the minimum *Rettungswegbreiten* prescribed in the same guideline, which allows an evacuation capacity of roughly 100 persons per minute. Together, these constraints ensure that Scenario 1 remains both realistic in terms of safety standards and computationally tractable for repeated simulation.

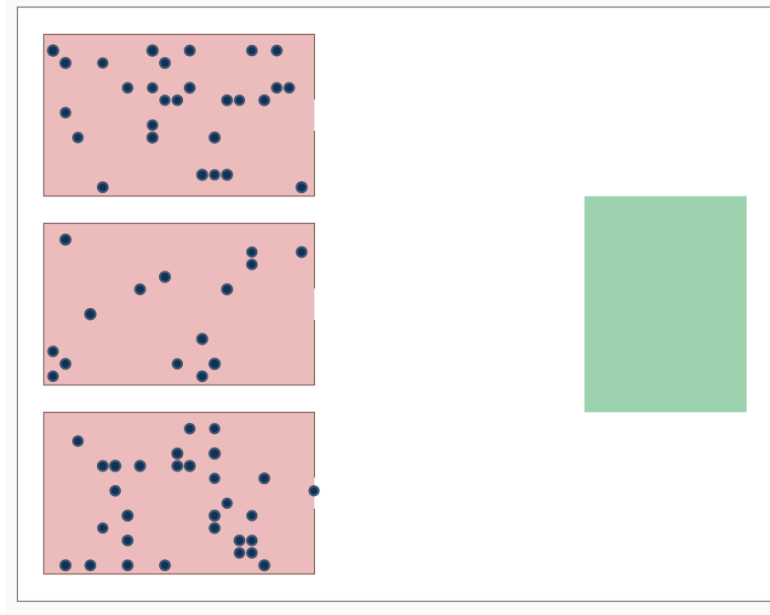


Figure 3.1: Scenario 1 geometry: Three origin rooms (red) evacuate to a common target area (green).

Dataset Generation

We executed 1,000 simulations per dataset. For each run, the room occupancies (N_1, N_2, N_3) were sampled independently at random from the integer range $[1, 30]$. The sampled occupancies were written to the input configuration (`intervals.xml`), the simulation was executed via the `crowd:it` kernel, and the resulting evacuation time was parsed from the output file (`sim.xml`). Each dataset therefore consists of tuples $(N_1, N_2, N_3, \text{EvacuationTime})$. Because of the simple geometry, Scenario 1 produces almost no congestion, so agents move without significant waiting times, and evacuation duration scales mainly with total population size. Beyond occupancy variation, datasets were generated by varying agent walking speeds and premovement delays, as described in the next subsection.

Behavioral Assumptions (Datasets)

While the geometry remained fixed, agent behavior was varied to create six distinct datasets (Datasets 2–7):

These choices span increasing behavioral variability: from homogeneous motion (D2–D3), to heterogeneity in walking speeds (D4–D5), and variability in start times

Table 3.1: Scenario 1 datasets (D2-D7) with varied behavioral conditions.

Dataset	Condition
D2, D3	All agents walk with fixed speed of 1.34 m/s, no premovement delay.
D4, D5	Agents have heterogeneous walking speeds drawn from a normal distribution $\mathcal{N}(\mu = 1.34, \sigma = 0.26)$ m/s, truncated to $[0.46, 1.61]$ m/s, with no premovement delay.
D6, D7	All agents walk with the same fixed speed of 1.34 m/s, but departure times include premovement delays drawn from a log-normal distribution $(\mu = 2, \sigma = 2)$ s, truncated to $[0, 10]$ s.

(D6–D7).

Dataset Quality Heuristic

Not all generated datasets were equally suitable for surrogate modeling. To filter out uninformative cases, we applied a simple quality heuristic based on the variance of evacuation times in the held-out test set, $\text{Var}(y_{\text{test}})$. If evacuation times vary little across simulations, the target values are nearly constant and the learning problem becomes trivial. Conversely, a broader spread indicates that the dataset captures meaningful behavioral variation and is suitable for training.

This criterion was used to decide which datasets were retained for further experiments. Datasets with insufficient variance were discarded, while D2–D7 were kept, as they all showed sufficient dispersion of evacuation times. Their approximate test-set variances were:

- Dataset 2: $\text{Var}(y_{\text{test}}) \approx 0.99$
- Dataset 3: $\text{Var}(y_{\text{test}}) \approx 1.21$
- Dataset 4: $\text{Var}(y_{\text{test}}) \approx 1.10$
- Dataset 5: $\text{Var}(y_{\text{test}}) \approx 1.03$
- Dataset 6: $\text{Var}(y_{\text{test}}) \approx 1.02$
- Dataset 7: $\text{Var}(y_{\text{test}}) \approx 1.02$

These values should be interpreted with care because the target variable is standardized before modeling, which causes variances across splits to cluster around 1. Here, we report $\text{Var}(y_{\text{test}})$ as a simple proxy, but the intent is to check that the underlying dataset exhibits non-trivial variation in evacuation times. This ensures that retained datasets are informative and can meaningfully support surrogate modeling, while degenerate cases were excluded.

3.1.2 Model Design & Training

The surrogate models in Scenario 1 were implemented using the SWIM pipeline, with up to three Dense layers followed by a final Linear output layer (see Figure 3.2). Across all experiments, the following hyperparameters were systematically varied:

Layer width: {4, 8, 16, 32, 64, 128, 256, 512}

Activation function: tanh, relu

Number of dense layers: 1, 2, or 3

Regularization scale: $10^{-6}, 10^{-8}$

Parameter sampler: SWIM-tanh, SWIM-relu, or random

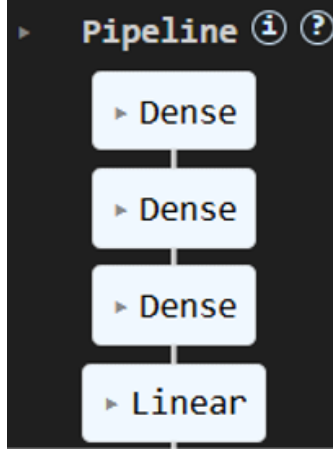


Figure 3.2: Example pipeline architecture: up to three Dense layers are stacked before a final Linear layer.

Feature Selection

Initial experiments included not only raw room occupancies (N_1, N_2, N_3) but also engineered features such as RMS, Euclidean norm, and Quadratic sum:

$$\text{Root-mean-square (RMS)} = \sqrt{\frac{1}{3} (N_1^2 + N_2^2 + N_3^2)},$$

$$\text{Euclidean norm} = \sqrt{N_1^2 + N_2^2 + N_3^2},$$

$$\text{Quadratic sum} = N_1^2 + N_2^2 + N_3^2.$$

As shown in Figure 3.3, these engineered features are highly correlated with the raw inputs and with each other. Since they contributed little additional independent information while increasing runtime, the final models used only (N_1, N_2, N_3) .

Parameter Sampling

In the SWIM framework, parameter sampling is not random but data-driven. Instead of drawing weights and biases from uninformed Gaussian or uniform distributions, SWIM

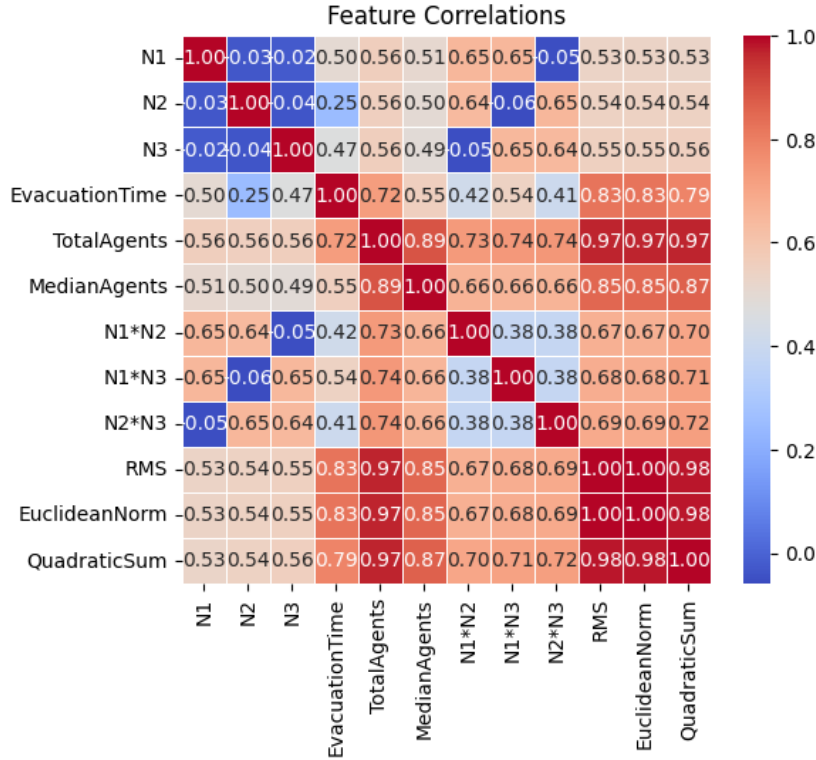


Figure 3.3: Correlation matrix of Scenario 1 features. Engineered variables (RMS, EuclideanNorm, RoomLoadRange) are highly collinear with the raw room occupancies (N_1, N_2, N_3) and therefore add little independent information.

samples them from pairwise differences of training points, aligning network parameters with meaningful variations in the input-output space (Bolager, Burak, Datar, et al., 2023). For computational efficiency, only a random subset of all possible pairs is used. This ensures that sampled neurons are well-matched to the structure of the data. In contrast, random initialization disregards this structure, which often results in unstable or poor fits in our experiments. As later results demonstrate, SWIM samplers (tanh, relu) consistently produced higher and more stable R^2 values compared to random sampling.

Preprocessing, Splits, and Evaluation

Before splitting, both input features (N_1, N_2, N_3) and the target variable (evacuation time) were standardized using the StandardScaler from scikit-learn (Pedregosa,

Varoquaux, Gramfort, et al., 2011). Standardization centers each variable to zero mean and scales it to unit variance, ensuring comparability across dimensions and stabilizing neural training.

All datasets were then split into training (80%), validation (10%), and test (10%). Performance was evaluated using mean squared error (MSE) and the coefficient of determination (R^2).

For reproducibility and further analysis, trained models were serialized using `joblib` (Joblib Development Team, 2023) and stored as `.pkl` files, allowing best-performing configurations to be reloaded without retraining.

3.1.3 Results

Performance across Datasets

Table 3.2 summarizes results for all six datasets (D2–D7). Each row reports the variance of y_{test} , the best R^2 achieved with its corresponding configuration, and the median R^2 across all hyperparameter settings.

Table 3.2: Scenario 1 summary across Datasets 2-7. Reported are the dataset’s evacuation-time variance, the best validation R^2 , the median validation R^2 across all hyperparameter settings, and the configuration(width, activation, #Dense, reg) of the best model.

Dataset	Condition	Var(y_{test})	Best R^2	Median R^2	Best Config
2	Same speed (homog.)	0.99	0.870	0.832	32, tanh, 1, 10^{-6}
3	Same speed (homog.)	1.21	0.894	0.851	128, relu, 2, 10^{-6}
4	Normal speed (heterog.)	1.10	0.490	0.406	256, tanh, 1, 10^{-8}
5	Normal speed (heterog.)	1.03	0.399	0.327	128, tanh, 1, 10^{-8}
6	Premovement log-normal + same speed	1.02	0.673	0.494	32, tanh, 1, 10^{-6}
7	Premovement log-normal + same speed	1.02	0.679	0.604	32, tanh, 1, 10^{-6}

Note: “#Dense” is the number of Dense layers before the Linear output layer.

Figure 3.4 visualizes the distribution of validation R^2 values. Predictive accuracy is highest for homogeneous speed datasets (2, 3), substantially reduced under het-

erogeneous walking speeds (4, 5), and intermediate when premovement delays are introduced (6, 7).

To illustrate individual model performance, Figure 3.5 plots predicted versus actual evacuation times for Dataset 3, showing tight alignment around the diagonal and thus confirming the high accuracy achieved in the homogeneous case.

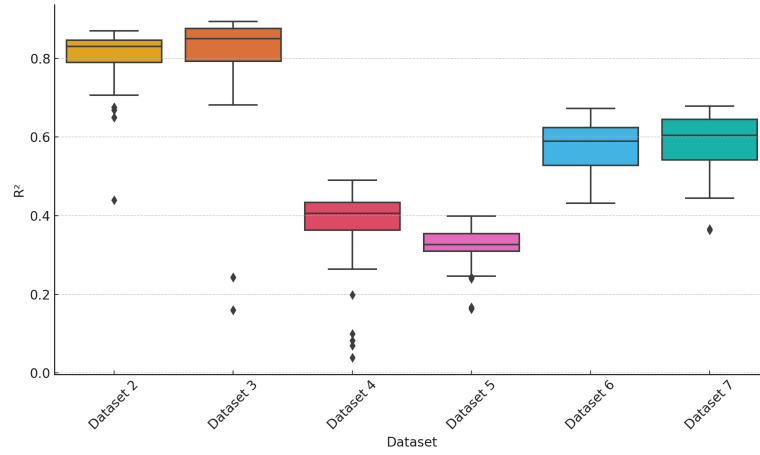


Figure 3.4: Validation R^2 distributions across Datasets 2–7 shown.

SWIM vs. Random Parameter Sampling

To compare initialization strategies, we evaluated SWIM samplers (tanh, relu) against uninformed random sampling. For each dataset, we took the mean of the top three positive validation R^2 values. This avoids skew from unstable random runs (negative R^2) while ensuring a fair comparison across both methods.

Figure 3.6 summarizes the comparison. In the homogeneous datasets (D2–D3), SWIM achieves mean R^2 scores of 0.87 and 0.89, nearly perfect fits, whereas random initialization drops to 0.52 in D2 and only approaches SWIM’s level in D3 (0.86). This highlights how random sampling can occasionally succeed but lacks robustness.

In the heterogeneous walking-speed datasets (D4–D5), the gap becomes striking and SWIM maintains moderate accuracy (0.48 and 0.39), while random initialization collapses to near-zero (0.07 and 0.20). This illustrates the instability of uninformed initialization under complex behavioral variation.

Finally, in the premovement-delay datasets (D6–D7), SWIM again secures stable performance around 0.67–0.68, while random initialization lags far behind (0.24 and 0.25). The consistency of SWIM across conditions contrasts with the erratic and generally weaker outcomes of random sampling.

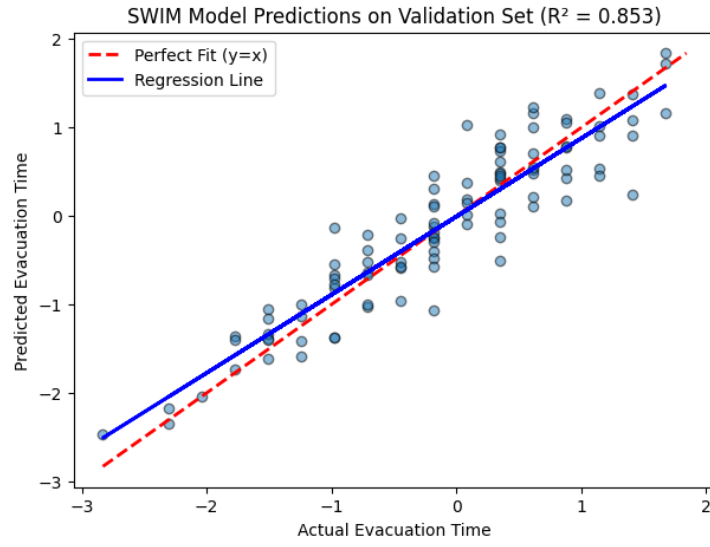


Figure 3.5: Validation set predictions vs. ground-truth evacuation times (Dataset 3) shown.

Overall, the figure demonstrates that data-aligned parameter sampling not only improves mean predictive accuracy but also reduces the volatility of results across datasets.

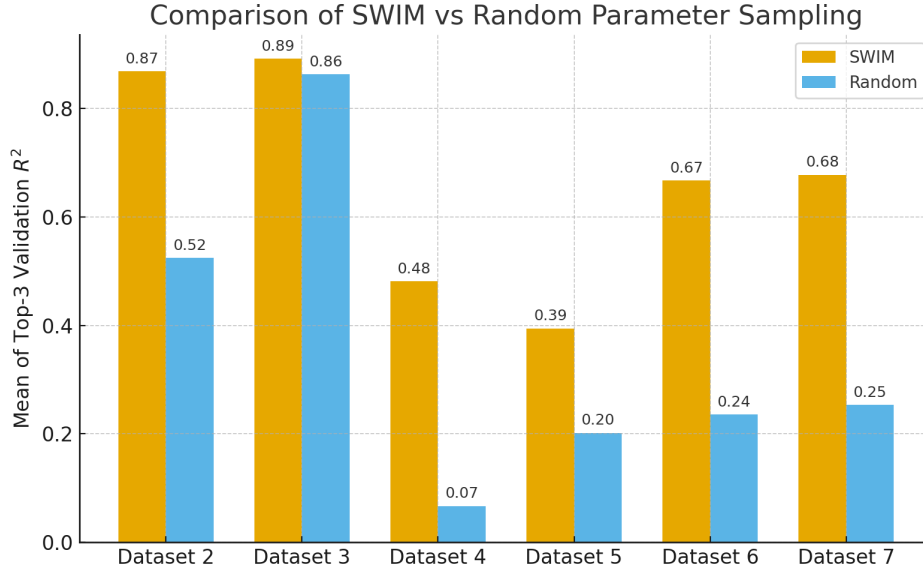


Figure 3.6: Mean of the top three positive validation R^2 values for each dataset. SWIM-based parameter sampling consistently outperforms random initialization, yielding higher predictive accuracy and greater stability.

Feature Engineering Results

To test whether engineered features could improve predictive accuracy, we extended the raw input set (N_1, N_2, N_3) with various combinations such as RMS, RoomLoadRange, EuclideanNorm, MeanRoomLoad, MaxRoomLoad, QuadraticSum, and interaction terms. Representative results for Dataset 2 are shown in Figure 3.7.

Most engineered feature sets achieved validation R^2 values in the range 0.886–0.898, slightly higher than the best raw model under the restricted comparison ($R^2 = 0.87$).[†] In particular, adding *RoomLoadRange* or combinations of *RMS* and *EuclideanNorm* yielded the strongest improvements. However, these gains were marginal (at most $\Delta R^2 \approx 0.03$), and all engineered features were highly collinear with the raw occupancies, as already shown in the correlation analysis (Figure 3.3).

We therefore conclude that engineered features offer only limited benefit in Scenario 1. Although some combinations (e.g., RMS, RoomLoadRange) slightly improved validation R^2 by up to $\Delta R^2 \approx 0.03$, these gains were not justified given the strong collinearity with the raw occupancies and the additional preprocessing complexity. The final models therefore retained only (N_1, N_2, N_3) as inputs, consistent with Scenario 1’s role as a simplified baseline.

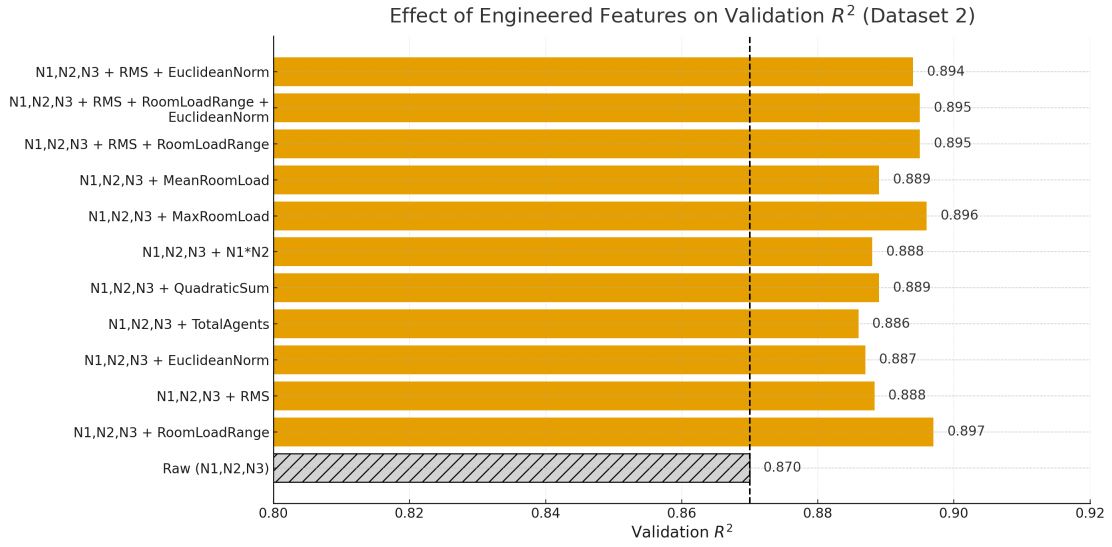


Figure 3.7: Effect of engineered features on validation R^2 in Dataset 2. Marginal improvements are observed (up to $\Delta R^2 \approx 0.03$), but raw room occupancies (N_1, N_2, N_3) already capture the main predictive signal.

Note: For a fair comparison, only hyperparameter configurations shared across raw and engineered feature sets were considered. Under this restriction, the best raw model reached $R^2 = 0.87$. In the full sweep of raw inputs, however, top models achieved $R^2 \approx 0.89$, matching the engineered-feature runs.

3.1.4 Discussion

Scenario 1 results should be interpreted in light of its highly simplified geometry and behavioral assumptions: Agents evacuate without congestion, so evacuation times scale almost linearly with total occupancy. This explains why homogeneous datasets (2-3) achieved R^2 near 0.9, because the mapping is nearly deterministic. Introducing heterogeneity (walking speeds or premovement delays) injected stochasticity, reducing predictability.

Key findings:

- Surrogate modeling is highly effective in simplified, congestion-free settings, with R^2 close to 0.9.
- Behavioral variability decreases predictability: heterogeneous speeds and delays lowered R^2 to the 0.3-0.6 range.
- SWIM samplers consistently outperformed random initialization, highlighting

the value of data-aligned parameter sampling.

- Feature engineering offered little benefit and was excluded from the final models, as the engineered variables were highly collinear with the raw inputs and yielded only marginal improvements (at most $\Delta R^2 \approx 0.03$).

Thus, Scenario 1 serves as a baseline, validating the approach under controlled conditions, but does not represent realistic evacuation dynamics. This motivated Scenario 2, where architectural factors and more complex behaviors are introduced.

3.2 Scenario 2: Vertical Evacuation Prediction

3.2.1 Data Preparation

Scenario 2 models vertical evacuations in multi-storey buildings. Each simulation is parameterized by three architectural and demographic variables:

- E : number of floors,
- T : stair width (cm),
- P : number of people per floor.

In addition, each parameter setting was run multiple times to account for stochastic variation, with `RunNo` serving as the replicate identifier. The prediction target is the total evacuation time, measured as the `LastStep` of the simulation.

While the raw `-stat.csv` files contain many microscopic statistics (e.g., per-floor densities, flow rates), we retained only $(E, T, P, \text{RunNo}, \text{LastStep})$, consistent with prior studies highlighting E , T , and P as the key architectural and demographic drivers of vertical evacuation times (Kneidl & Könnecke, 2020). This focuses the learning task on macroscopic evacuation duration, while discarding intermediate measures not directly relevant for surrogate modeling.

The assembled dataset (`combined_vertical_evacuations.csv`) contains all runs across parameter combinations, producing a total of 17,410 samples derived from the Gd99 study. This provides a rich empirical basis for surrogate modeling and also serves as a reference point for later sampling strategies, where we demonstrate that comparable predictive accuracy ($R^2 \approx 0.95$) can be achieved with fewer than ten carefully chosen simulations, amounting to only about 0.05% of the full dataset.

Figure 3.8 shows the distribution of `LastStep`, with most evacuations completing between 200 and 400 steps, but a long tail extending beyond 1,000 steps in challenging scenarios.

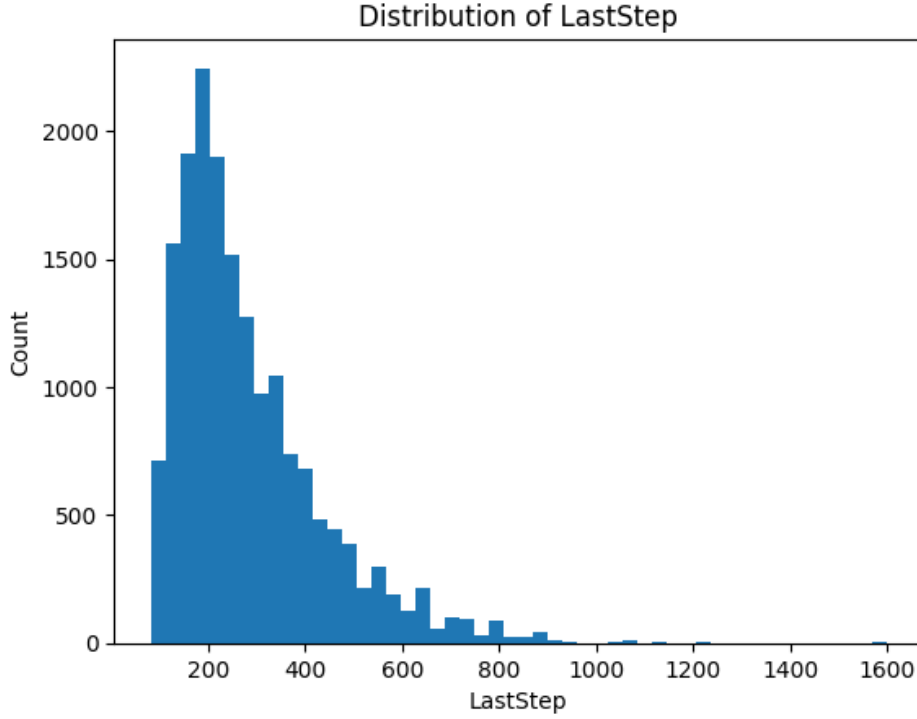


Figure 3.8: Distribution of evacuation times (LastStep) in Scenario 2. Most evacuations finish between 200 and 400 steps, but outliers extend far longer.

Predictive Power of Inputs

To assess how much variance can be explained by the architectural parameters alone, we fitted baseline linear regression models using `scikit-learn` (Pedregosa, Varoquaux, Gramfort, et al., 2011) on subsets of (E, T, P) . Inputs and targets were standardized, and the dataset was split into 80% training, 10% validation, and 10% test sets. Models were trained on the respective subsets and evaluated using the coefficient of determination (R^2) on the held-out test split. Figure 3.9 summarizes the results.

The results show that P (people per floor) is the strongest individual predictor, followed by E (floors), while T (stair width) alone has weak explanatory power. However, all three parameters together are required to approach high accuracy ($R^2 \approx 0.83$). This indicates that evacuation time is shaped by the joint interaction of population load (P), vertical distance (E), and stair width (T). It also demonstrates that the dataset is intrinsically predictive, even with simple linear models, motivating the use of neural networks to capture non-linear interactions.

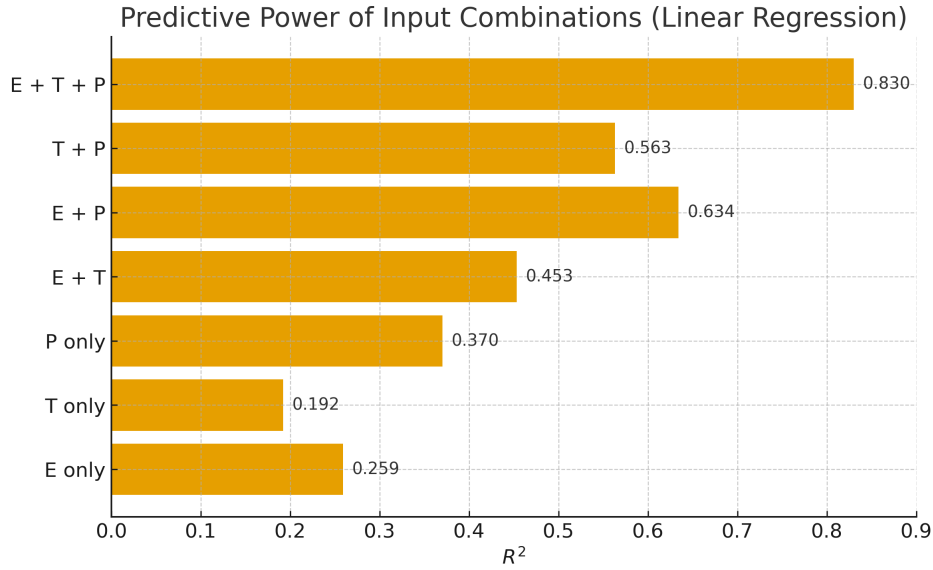


Figure 3.9: Baseline regression R^2 for different input subsets. The full set (E, T, P) clearly outperforms single-parameter models, underscoring the importance of interaction effects.

3.2.2 Model Design & Training

The SWIM pipeline was again employed, with up to three Dense layers followed by a Linear output layer. Hyperparameters varied were:

- Layer width: {4, 8, 16, 32, 64, 128, 256, 512},
- Number of Dense layers: 1–3,
- Activation: relu, tanh,
- Regularization: 10^{-6} , 10^{-7} , 10^{-8} ,
- Parameter sampler: aligned with activation (relu/tanh) or uninformed (random).

Both the input variables (E, T, P) and the target variable (LastStep) were standardized using separate StandardScaler instances from scikit-learn (Pedregosa, Varoquaux, Gramfort, et al., 2011). Each was transformed to zero mean and unit variance. This improves comparability across variables and stabilizes neural training.

The dataset was then partitioned into training (80%), validation (10%), and test (10%) subsets. Model performance was evaluated with the coefficient of determination (R^2) and mean squared error (MSE), consistent with Scenario 1.

3.2.3 Results

Validation Performance Across Hyperparameters

Validation performance was analyzed across depth, activation, regularization, and layer width. Figures 3.10–3.13 summarize the results.

Figure 3.10 shows the effect of network depth. A single hidden layer already delivers strong accuracy with tight variance across runs. Adding more layers (2-3) does not yield consistent improvements and instead produces a broader spread of R^2 values, including some markedly lower outliers.

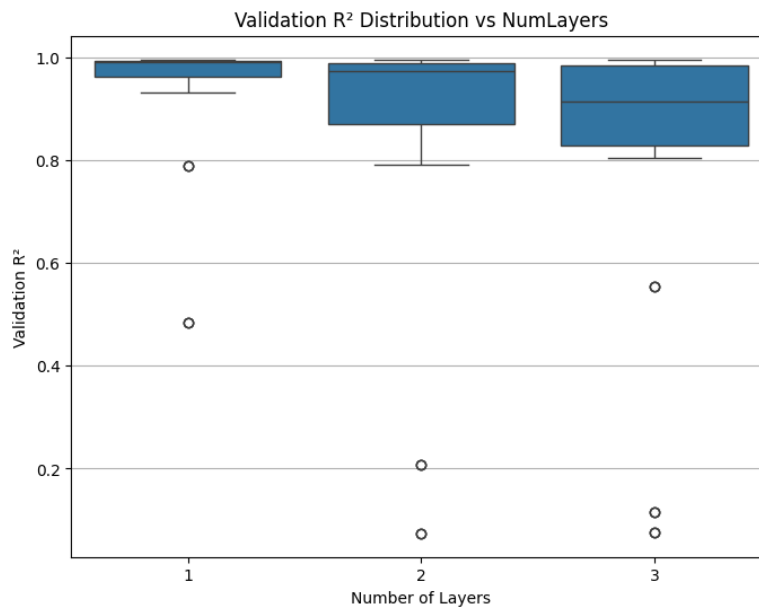


Figure 3.10: Validation R^2 distribution across different network depths. A single hidden layer achieves high accuracy with low variance. Additional layers (2-3) do not systematically improve performance and lead to a wider spread of outcomes.

Figure 3.11 compares activations. Both `relu` and `tanh` reach high accuracy, but `ReLU` shows slightly higher medians and fewer extreme outliers, indicating greater robustness in this setting.

The effect of regularization (Figure 3.12) is minimal across 10^{-6} to 10^{-8} , validation R^2 remains stable, suggesting that overfitting is not a major concern given the dataset size and the simplicity of the task.

By contrast, layer width has a pronounced impact (Figure 3.13). Performance im-

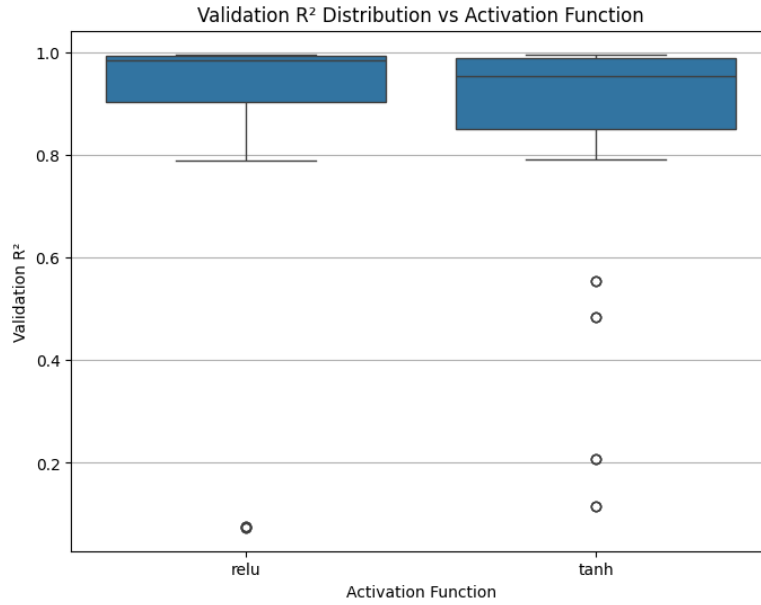


Figure 3.11: Validation R^2 distribution across activation functions. ReLU yields slightly higher medians and fewer outliers than tanh, though both activations can reach high accuracy.

proves consistently as width increases, with plateauing gains beyond 256 units. Wider networks thus provide better predictive capacity, but with diminishing returns once sufficient representational power is reached.

The joint effect of depth and width is summarized in Figure 3.14. Very small widths (4-8) consistently underperform, while all configurations with ≥ 64 units achieve average scores above $R^2 = 0.90$. Some cells appear as $R^2 = 1.00$ due to rounding. In practice, the highest score observed was $R^2 \approx 0.997$. Results show that performance is governed primarily by layer width, while depth provides only marginal gains and regularization contributes little.

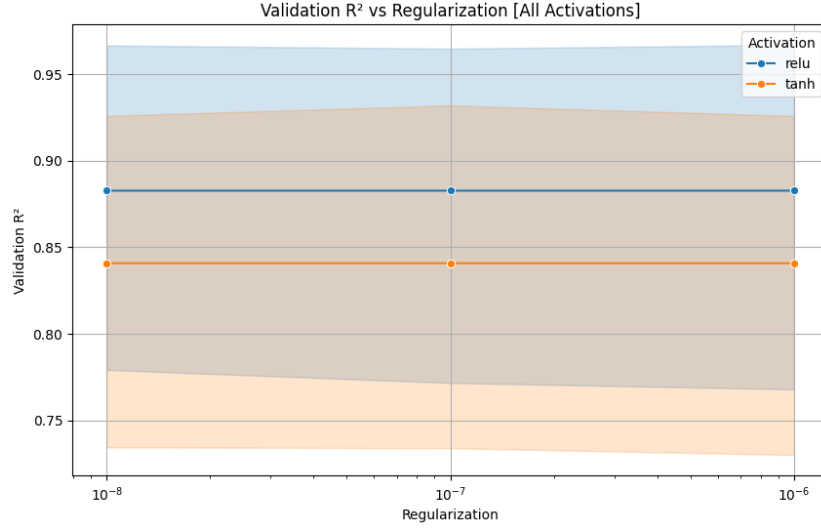


Figure 3.12: Validation R^2 across regularization scales. Performance is stable between 10^{-6} and 10^{-8} , with little sensitivity to this parameter.

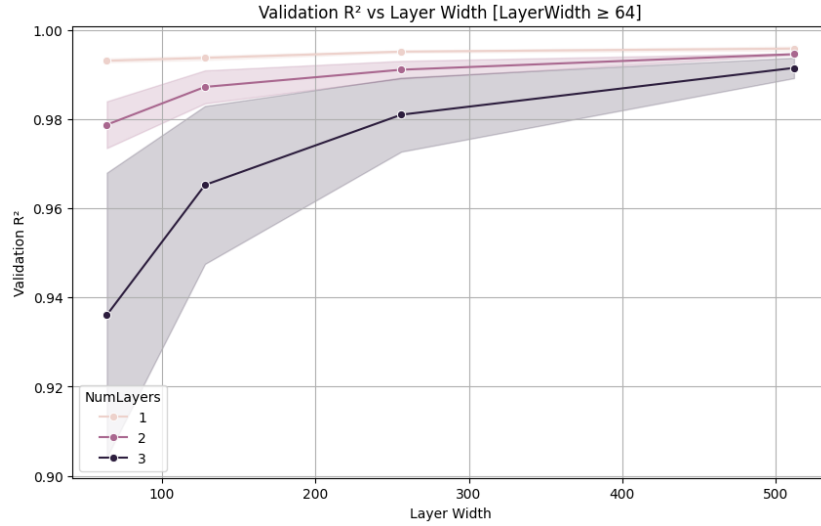


Figure 3.13: Validation R^2 vs layer width (for widths ≥ 64). Wider networks improve predictive accuracy, plateauing near $R^2 \approx 0.99$.

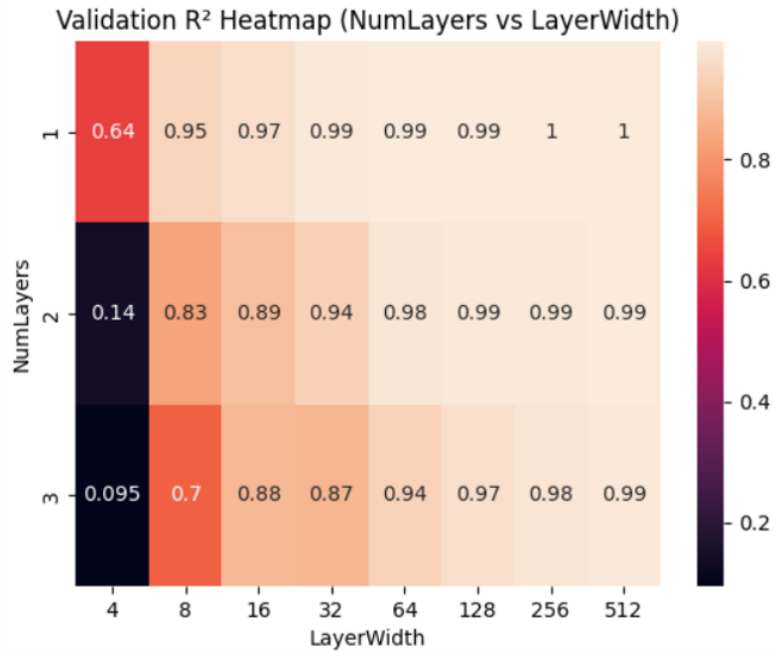


Figure 3.14: Mean validation R^2 across layer width and depth (regularization fixed at 10^{-6}). Width dominates performance, while depth has only a minor effect. Values rounded to 1.00 indicate $R^2 \geq 0.995$.

Train-Validation Consistency

Figure 3.15 compares train and validation R^2 across all hyperparameter settings. Nearly all points lie close to the diagonal, indicating that models generalize well and that overfitting is minimal.

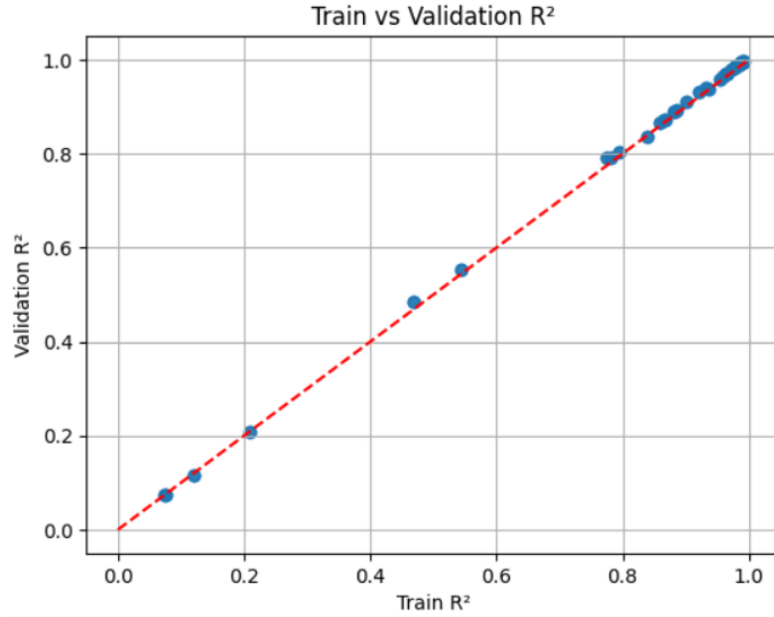


Figure 3.15: Train vs validation R^2 across hyperparameter settings. Points close to the diagonal confirm minimal overfitting and strong generalization.

Best Model and Test Evaluation

The best-performing configuration used tanh activations, a single Dense layer of width 512, and 10^{-8} regularization, with the parameter sampler also set to tanh. Despite its shallow depth, this wide network achieved the strongest results. Figure 3.16 shows the validation predictions, which closely follow the diagonal with $R^2 \approx 0.997$.

When retrained on the combined training and validation data and evaluated on the held-out test set, the same model achieved $R^2 = 0.993$ (Figure 3.17). Predictions again aligned closely with ground truth, confirming that the surrogate model generalizes well to unseen data and can accurately capture vertical evacuation dynamics.

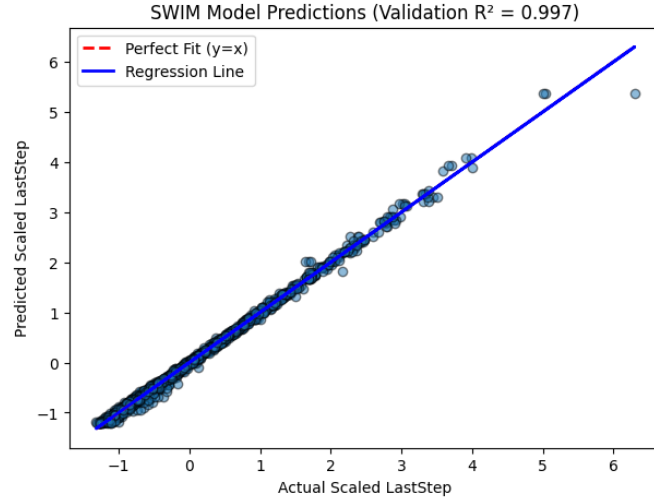


Figure 3.16: Validation set predictions vs. ground truth for the best configuration(tanh, width 512, 1 layer, 10^{-8} reg.). Predictions align almost perfectly with ground truth ($R^2 \approx 0.997$).

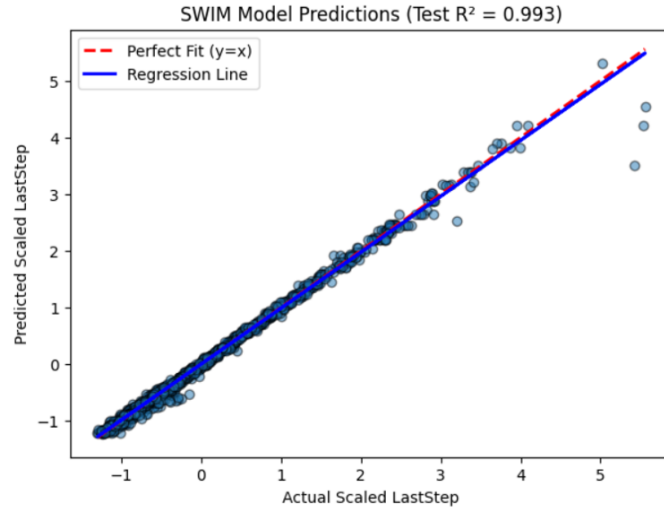


Figure 3.17: Test set predictions vs ground truth. Diagonal alignment ($R^2 \approx 0.993$) confirms the model generalizes to unseen data.

SWIM vs. Vanilla NN: Accuracy and Speed

We benchmarked SWIM against a vanilla feed-forward network implemented in PyTorch (Paszke, Gross, Massa, et al., 2019), using the same $(E, T, P) \rightarrow \text{LastStep}$ splits. This comparison matters because when many hyperparameter configurations must be tested, wall-clock training time per fit is just as important as final accuracy. A method that reaches the same R^2 substantially faster is better suited for iterative design and sampling studies.

Table 3.3: Best accuracy and typical training time per fit across methods (Scenario 2). Best rows are selected by validation R^2 ; training time is the median over runs.

Method	Act.	Layers	Width	Reg.	Best Val R^2	Best Test R^2	Med. Train (s)
SWIM (Rand.)	tanh	1	512	10^{-8}	0.997	0.993	0.03
SWIM (Act.)	relu	2	512	10^{-6}	0.997	0.993	—
Vanilla NN	tanh	2	512	10^{-6}	0.996	0.992	1.43

- “Act.” = activation-aligned parameter sampling with uniform draws (sample_uniformly=True).
- All methods were evaluated on the same hyperparameter grid as in Section 3.2.2 (layer widths up to 512, depths up to 3, tanh/relu, and regularizations 10^{-6} - 10^{-8}), ensuring a fair comparison of accuracy and training time.
- Training time is measured per fit; activation-uniform runs did not log time (“—”).
- All methods converge to essentially the same test accuracy ($R^2 \approx 0.99$).

Across all methods, accuracy was nearly identical. However, SWIM required far less training time. The activation-aligned variant did not improve peak accuracy but reduced variance across runs (std. dev. of validation $R^2 = 0.003$ vs. 0.012 for random), making it more robust. Vanilla NN matched accuracy but was substantially slower. On average, SWIM trained about **50× faster** (0.03 s vs. 1.43 s per fit), highlighting its clear advantage for large-scale hyperparameter searches.

Figure 3.18 shows that both methods achieve comparable accuracy, with median

validation scores close to $R^2 \approx 0.99$. SWIM exhibits slightly greater variance, including occasional outliers, while vanilla NNs are more tightly concentrated.

Training time, however, reveals a stark difference (Figure 3.19). While vanilla NNs required several seconds to train, SWIM consistently completed in a fraction of a second.

To visualize the trade-off between accuracy and runtime, Figure 3.20 plots R^2 against training time. SWIM dominates the lower-left corner, achieving high accuracy almost instantly, whereas vanilla NNs require much longer training to approach the same scores.

Efficiency, measured as R^2 per second of training, further highlights the contrast (Figure 3.21). Here, SWIM outperforms vanilla NNs by more than an order of magnitude. This advantage became more visible when restricting to the top-40 hyperparameter configurations by validation accuracy (Figure 3.22), demonstrating that SWIM retains efficiency not only across the full grid but also among the strongest models.

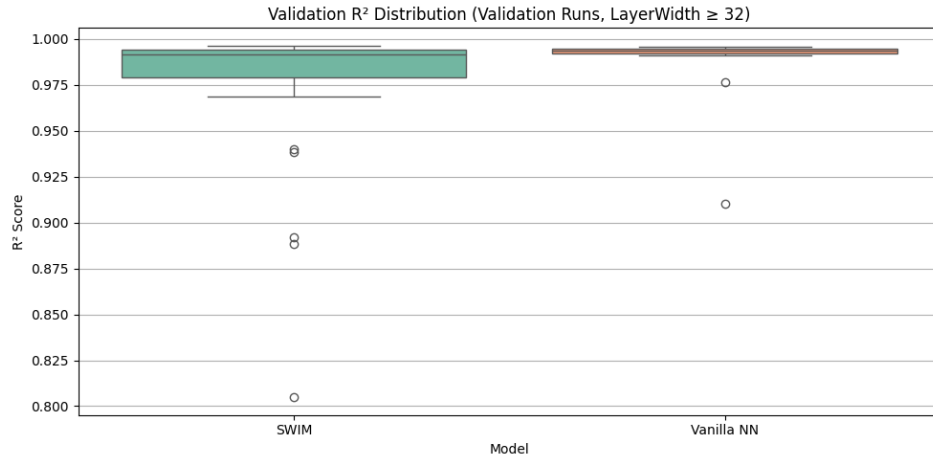


Figure 3.18: Validation R^2 distribution (layer width ≥ 32). Both SWIM and vanilla NNs reach similar medians near $R^2 \approx 0.99$, though SWIM shows slightly higher variance.

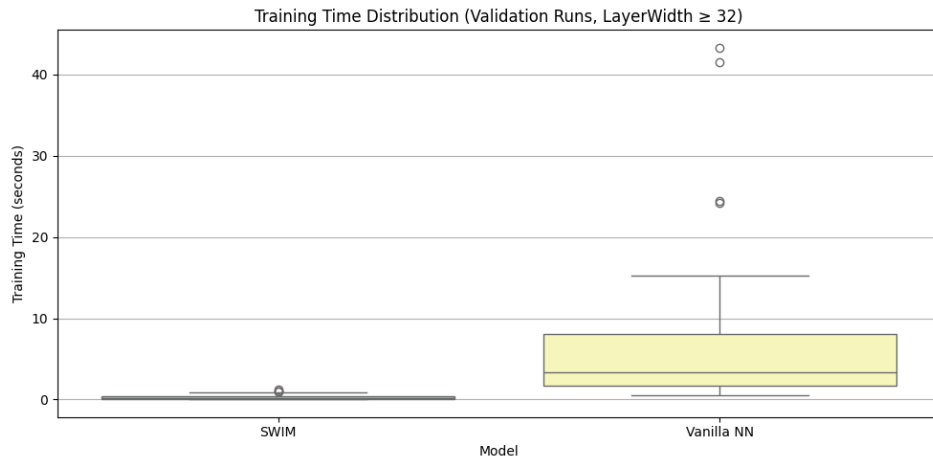


Figure 3.19: Training time distribution (layer width ≥ 32). SWIM trains around $50\times$ faster than vanilla NNs.

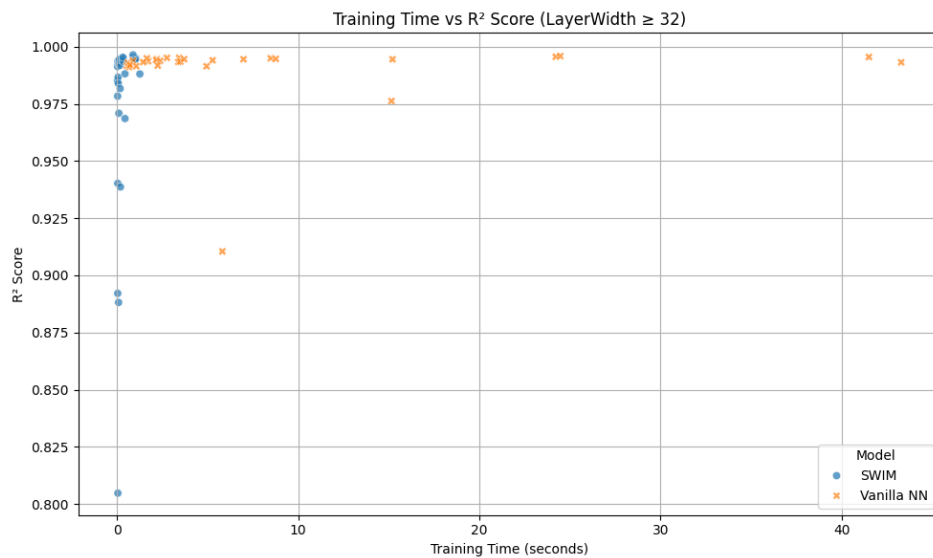


Figure 3.20: Validation R^2 vs training time (layer width ≥ 32). SWIM achieves high accuracy almost instantly, while vanilla NNs require much longer runtimes to reach comparable scores.

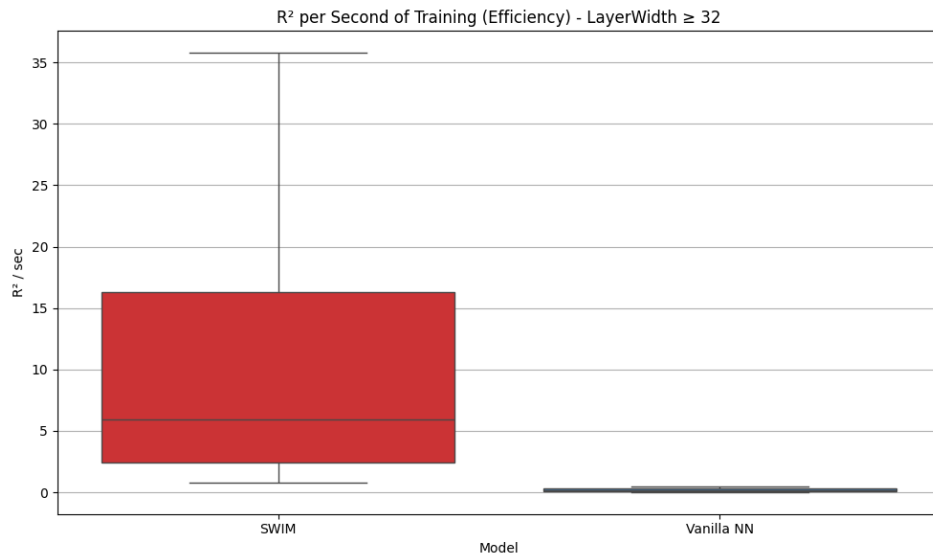


Figure 3.21: Efficiency measured as R^2 per second of training (layer width ≥ 32). SWIM achieves over an order of magnitude higher efficiency.

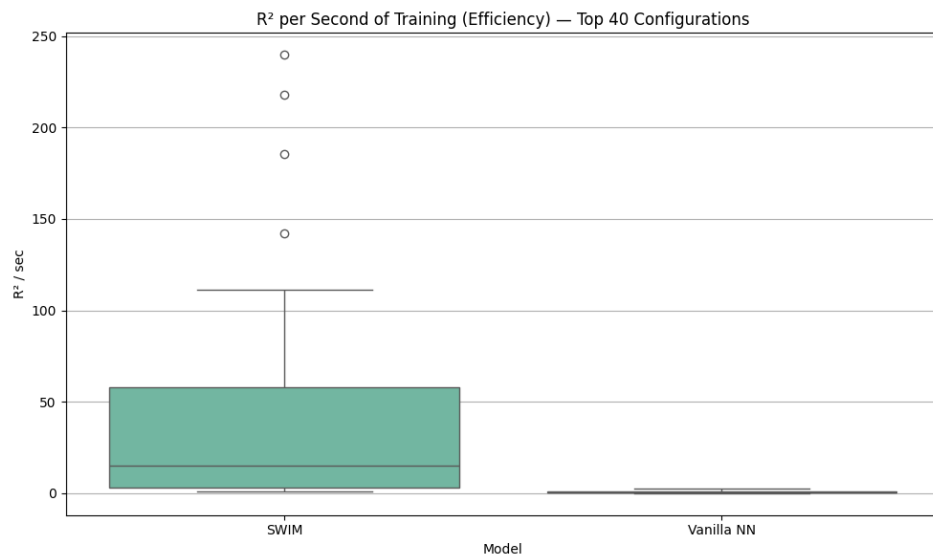


Figure 3.22: Efficiency comparison restricted to the top-40 configurations (by validation R^2). SWIM retains its efficiency advantage even among the strongest models.

3.2.4 Discussion

Scenario 2 yields several key insights:

- Even simple linear regressions reached $R^2 \approx 0.83$, indicating that the relationship between the model inputs and outputs is captured well by a linear function
- Neural networks pushed this further to test $R^2 > 0.99$, showing that non-linear interactions between (E, T, P) are critical to capture.
- Among the three inputs, population per floor (P) emerged as the strongest driver, while vertical distance (E) and stair capacity (T) were also indispensable to achieve high accuracy.
- Model width was the decisive architectural factor for stability and performance, while the effect of additional depth varied across configurations, sometimes improving and sometimes reducing accuracy. Regularization had only a marginal influence.
- Comparing methods, SWIM matched vanilla neural networks in accuracy but trained about $50\times$ faster, making it highly attractive for large-scale hyperparameter searches and simulation-driven sampling studies. The activation-aligned variant further reduced variance across runs, yielding more consistent results.
- Train-validation comparisons confirmed strong generalization, with little evidence of overfitting.

Unlike Scenario 1, which relied on a simplified and largely congestion-free geometry, Scenario 2 is grounded in realistic building layouts simulated with `crowd:it`. This makes it a practical benchmark that surrogate models can reliably predict evacuation times directly from architectural parameters. However, training these models still required more than 17,410 full simulations. To make the approach efficient in real-world applications, the next section introduces sampling strategies that aim to achieve the same predictive accuracy with only a small fraction of the simulation cost.

3.2.5 Scenario 2: Sampling Strategies

Motivation

Full `crowd:it` simulations are time-consuming and computationally costly. Sampling allows us to train surrogate models with a reduced number of simulations while maintaining predictive accuracy above 95% with only 0.05% of the full dataset. Figure 3.23 illustrates that instead of requiring the entire dataset, only a small subset of points is selected (red) while preserving broad coverage of the (E, T, P) parameter space.

Sampling Coverage in (E, T, P) Parameter Space

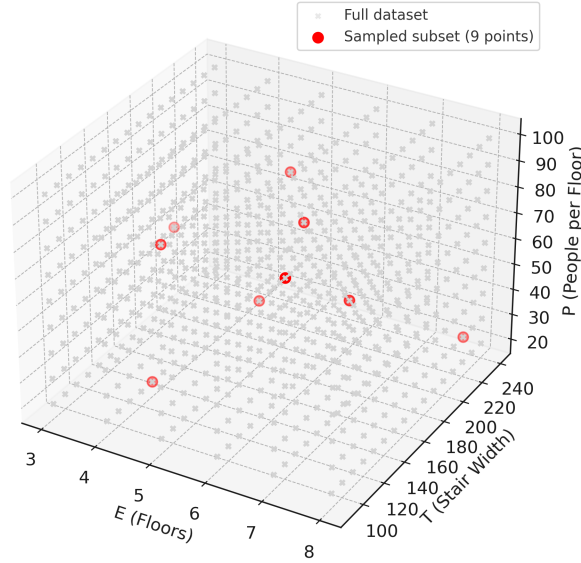


Figure 3.23: Sampling coverage in (E, T, P) parameter space. Grey points = full dataset; red points = sampled subset (9 points). Sampling reduces simulation cost while retaining representativeness.

Sampling Methods

We benchmarked several strategies for selecting representative parameter combinations (E, T, P) , each grounded in established theory:

- **Quasi-Monte Carlo (QMC):** QMC methods generate evenly spaced points across the parameter space, improving coverage compared to purely random sampling. Sobol' sequences are especially popular as low-discrepancy sequences for numerical integration and sampling tasks (Sobol', 1967).
- **Randomized QMC (RQMC):** RQMC extends QMC by applying scrambling to Sobol' sequences, breaking up regular patterns while preserving uniformity. Owen (Owen, 1998) showed that scrambled sequences can reduce error rates nearly to $\mathcal{O}(n^{-3/2})$, substantially outperforming both Monte Carlo and unsystematic QMC in many cases.
- **KMeans Sampling:** KMeans partitions the parameter space into clusters and selects the most representative point (centroid) from each cluster. This ensures that samples reflect the global structure of the space by minimizing within-cluster

variance and preventing oversampling of dense regions (Lloyd, 1982). To improve initialization and stability, variants such as KMeans++ have been proposed (Arthur & Vassilvitskii, 2006), which can yield more balanced clustering outcomes.

- **Entropy-Based Sampling:** This method prioritizes samples from parameter settings with the highest uncertainty in their outcome distribution. Shannon’s definition of entropy provides a quantitative measure of uncertainty, making it a natural criterion for informative sampling (Cover & Thomas, 1991; Shannon, 1948).
- **Sparse Grid Sampling:** Sparse grids cover the parameter space using a hierarchical grid structure, allowing high-dimensional domains to be approximated with relatively few points. By concentrating points in the most informative regions, sparse grids achieve high accuracy in moderate dimensions with dramatically fewer nodes than full tensor grids (Stoyanov, 2017).

Sampling and Experimentation Pipeline

The experimental pipeline mirrored the design of Scenario 2 but was applied to sampled subsets instead of the full dataset:

- Select (E, T, P, RunNo) combinations using a sampling method.
- Train SWIM-based neural networks on the sampled subsets using a hyperparameter search over layer widths (64-4096), depths (1-3), activations (relu, tanh), parameter sampler (random), and regularization (10^{-1} - 10^{-2}).
- Evaluate models on validation and test splits (10-20% each, depending on dataset size).
- Assess generalization on the unseen portion of the dataset, defined as the full dataset minus the sampled training set, using the best hyperparameters.

Results

Table 3.4 summarizes validation, test, and generalization R^2 for all sampling methods. Several methods surpassed the benchmark threshold of $R^2 > 0.95$ with as few as 9-14 samples.

Table 3.4: Comparison of sampling methods in Scenario 2. “# Samples” refers to the number of (E, T, P, RunNo) points used for training. Generalization R^2 is measured on the remainder of the dataset not seen during training.

Method	# Samples	Val R^2	Test R^2	Gen R^2
RQMC ($k=9$, RunNo=0)	9	0.991	0.994	0.950
RQMC ($k=14$, RunNo=0)	14	0.985	0.998	0.970
QMC ($k=10$, RunNo=(0,1))	20	0.943	0.997	0.945
Entropy ($k=12$, RunNo=0,2)	36	0.998	0.999	0.865
Entropy ($k=10$, RunNo=0,2)	30	0.998	0.984	0.949
KMeans ($k=5$, RunNo=0,3)	20	0.731	0.989	0.717
Sparse Grid ($d=2$, RunNo=0,2)	46	0.997	0.976	0.966

Note: Models with fewer than 7 samples were excluded, as validation and test splits would otherwise contain only one case, producing unstable and unreliable R^2 estimates.

Overfitting controls and rationale

We explicitly stress-tested regularization, because the sampled sets can be extremely small (e.g., 9-20 points). We expanded the search to *large* L_2 values ($\lambda \in \{10^{-2}, 10^{-1}\}$) and observed that (i) validation/test R^2 remained essentially unchanged and (ii) R^2_{gen} on $\mathcal{D}_{\text{gen}}$ stayed high (e.g., RQMC with $k=9$ achieved $R^2_{\text{gen}} \approx 0.95$; $k=14$ reached ≈ 0.97). This insensitivity to strong regularization, together with the large unseen evaluation set, indicates that the reported gains are not artifacts of overfitting on tiny samples.

Sampling Overlap Analysis

RQMC with 9 and 14 samples reached the best trade-off between cost and accuracy. The 9-sample set is a strict subset of the 14-sample set (Figure 3.24), which explains their nearly identical performance. The five additional points in the 14-sample case expand coverage only marginally without altering overall representativeness.

Overlap Between RQMC-9 and RQMC-14

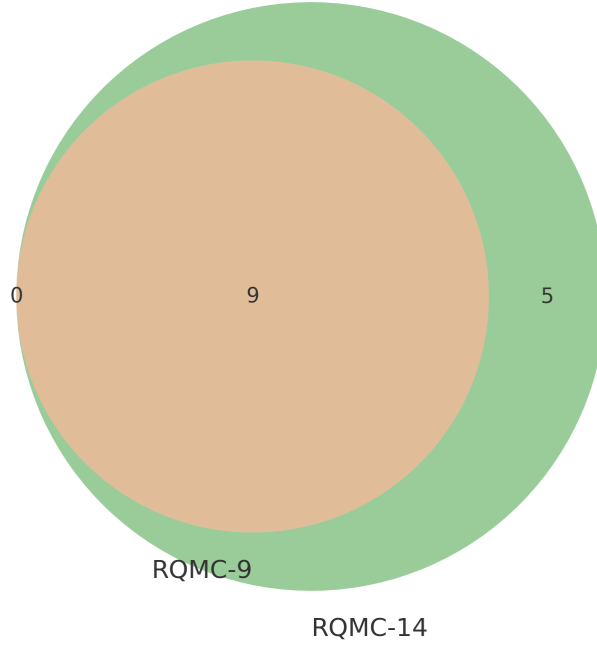


Figure 3.24: Overlap between RQMC-9 and RQMC-14 sampling sets. The 9-sample configuration is entirely contained within the 14-sample configuration.

Discussion

The results confirm that surrogate models can generalize well even with dramatically reduced training data. Among methods, RQMC emerged as the most effective with only 9-14 samples, it consistently achieved $R^2 > 0.95$ across validation, test, and generalization. QMC without scrambling was less robust, entropy sampling achieved high accuracy but at higher simulation cost, sparse grids were competitive but more expensive, and KMeans sampling proved unreliable. Overall, randomized low-discrepancy sequences (RQMC) strike the best balance between efficiency and predictive performance. These findings demonstrate that simulation costs in Scenario 2 can be reduced by more than an order of magnitude without compromising accuracy, enabling practical integration into iterative design workflows.

4 Conclusion and Future Work

This thesis explored the use of surrogate modeling to accelerate evacuation analysis in complex building layouts. While traditional microscopic simulations provide high fidelity, their computational cost hinders iterative design and large-scale parameter studies. To address this challenge, we applied the SWIM algorithm to train surrogate models on both synthetic and regulation-oriented datasets, investigating whether accurate evacuation predictions could be achieved with substantially fewer simulations.

The results of this thesis highlight three key findings. Firstly, with full training datasets, SWIM-based surrogates consistently reproduced macroscopic evacuation outcomes with very high accuracy, often achieving $R^2 > 0.99$. Secondly, sampling strategies played a decisive role in reducing cost. Randomized Quasi-Monte Carlo (RQMC) consistently delivered strong generalization with as few as 9-14 simulations, reaching R^2 values of 0.95-0.97. In contrast, methods such as entropy-based sampling or sparse grids required substantially larger sample sets to achieve comparable performance. Thirdly, validation across two complementary scenarios, namely a synthetic room-based setup and the regulation-derived Gd99 dataset, demonstrated that surrogate modeling is not only a methodological advance but also directly applicable to regulatory benchmarks and practical engineering contexts.

Together, these contributions confirm that reliable evacuation predictions can be obtained at a fraction of the simulation cost. This makes surrogate modeling a promising tool for both research and applied safety design, where rapid evaluations are often required under regulatory and practical constraints.

Although the thesis establishes a strong methodological basis, several avenues remain open for further development:

- **Cross-Scenario Testing:** Extend the evaluation of sampling and prediction methods to unseen building typologies, such as multi-use complexes or transport hubs, to test robustness beyond the scenarios studied here.
- **Integration into crowd:it:** Develop a seamless prediction workflow where users can directly select “Predict” within crowd:it. Minimal simulations would then be triggered automatically to train and cache surrogates, enabling instant predictions for future queries. This integration would allow practitioners to benefit from surrogate-based acceleration without additional manual steps.

- **Broader Sampling Studies:** While RQMC showed the strongest trade-off between efficiency and accuracy, further work could investigate adaptive schemes that refine sampling online based on uncertainty. Additionally, clustering-based approaches could be revisited using improved initialization methods such as KMeans++ (Arthur & Vassilvitskii, 2006), which may mitigate the instability observed with vanilla KMeans in this thesis.
- **Behavioral Extensions:** Future datasets could incorporate richer behavioral factors such as group cohesion, communication delays, or response-time distributions, extending the surrogate modeling framework to capture both geometric and behavioral variability.

By pursuing these directions, future research can move from methodological validation toward practical deployment, enabling surrogate modeling to become a standard component of evacuation analysis workflows in both academic and industrial contexts.

Abbreviations

AI	Artificial Intelligence
ASR	Workplace Ordinance (Arbeitsstättenrichtlinie)
Gd99	Guideline for Dimensioning of Crowd Flows at Events (1999)(Leitfaden zur Bemessung von Personenströmen bei Veranstaltungen (1999))
MBO	Model Building Code (Musterbauordnung)
MSE	Mean Squared Error
NN	Neural Network
QMC	Quasi-Monte Carlo
RQMC	Randomized Quasi-Monte Carlo
RiMEA	Guideline for Microscopic Evacuation Analysis (Richtlinie für Mikroskopische Entfluchtungs-Analysen)
SWIM	Sampling Where It Matters
TUM	Technical University of Munich (Technische Universität München)
BauA	Federal Institute for Occupational Safety and Health (Bundesanstalt für Arbeitsschutz und Arbeitsmedizin)

List of Figures

2.1	Conceptual model of occupant behavior in building fires: perception, interpretation, decision-making, and actions (Kuligowski, 2008).	5
2.2	Schematic of an agent-based framework, where autonomous entities interact with each other and their environment (Bandini, Manzoni, & Vizzari, 2009).	6
2.3	Local compression (left) and <i>crowd pressure</i> (right) in front of a bottleneck, illustrating congested and potentially hazardous zones near exits (Moussaïd, Helbing, & Theraulaz, 2011).	6
2.4	Illustration of the SWIM algorithm	8
3.1	Scenario 1 geometry: Three origin rooms (red) evacuate to a common target area (green).	11
3.2	Example pipeline architecture: up to three Dense layers are stacked before a final Linear layer.	14
3.3	Correlation matrix of Scenario 1 features. Engineered variables (RMS, EuclideanNorm, RoomLoadRange) are highly collinear with the raw room occupancies (N_1, N_2, N_3) and therefore add little independent information.	15
3.4	Validation R^2 distributions across Datasets 2–7 shown.	17
3.5	Validation set predictions vs. ground-truth evacuation times (Dataset 3) shown.	18
3.6	Mean of the top three positive validation R^2 values for each dataset. SWIM-based parameter sampling consistently outperforms random initialization, yielding higher predictive accuracy and greater stability.	19
3.7	Effect of engineered features on validation R^2 in Dataset 2. Marginal improvements are observed (up to $\Delta R^2 \approx 0.03$), but raw room occupancies (N_1, N_2, N_3) already capture the main predictive signal.	20
3.8	Distribution of evacuation times (LastStep) in Scenario 2. Most evacuations finish between 200 and 400 steps, but outliers extend far longer.	22
3.9	Baseline regression R^2 for different input subsets. The full set (E, T, P) clearly outperforms single-parameter models, underscoring the importance of interaction effects.	23

3.10	Validation R^2 distribution across different network depths. A single hidden layer achieves high accuracy with low variance. Additional layers (2-3) do not systematically improve performance and lead to a wider spread of outcomes.	24
3.11	Validation R^2 distribution across activation functions. ReLU yields slightly higher medians and fewer outliers than $\tanh$, though both activations can reach high accuracy.	25
3.12	Validation R^2 across regularization scales. Performance is stable between 10^{-6} and 10^{-8} , with little sensitivity to this parameter.	26
3.13	Validation R^2 vs layer width (for widths ≥ 64). Wider networks improve predictive accuracy, plateauing near $R^2 \approx 0.99$	26
3.14	Mean validation R^2 across layer width and depth (regularization fixed at 10^{-6}). Width dominates performance, while depth has only a minor effect. Values rounded to 1.00 indicate $R^2 \geq 0.995$	27
3.15	Train vs validation R^2 across hyperparameter settings. Points close to the diagonal confirm minimal overfitting and strong generalization. . .	28
3.16	Validation set predictions vs. ground truth for the best configuration ($\tanh$, width 512, 1 layer, 10^{-8} reg.). Predictions align almost perfectly with ground truth ($R^2 \approx 0.997$).	29
3.17	Test set predictions vs ground truth. Diagonal alignment ($R^2 \approx 0.993$) confirms the model generalizes to unseen data.	29
3.18	Validation R^2 distribution (layer width ≥ 32). Both SWIM and vanilla NNs reach similar medians near $R^2 \approx 0.99$, though SWIM shows slightly higher variance.	31
3.19	Training time distribution (layer width ≥ 32). SWIM trains around $50\times$ faster than vanilla NNs.	32
3.20	Validation R^2 vs training time (layer width ≥ 32). SWIM achieves high accuracy almost instantly, while vanilla NNs require much longer run-times to reach comparable scores.	32
3.21	Efficiency measured as R^2 per second of training (layer width ≥ 32). SWIM achieves over an order of magnitude higher efficiency.	33
3.22	Efficiency comparison restricted to the top-40 configurations (by validation R^2). SWIM retains its efficiency advantage even among the strongest models.	33
3.23	Sampling coverage in (E, T, P) parameter space. Grey points = full dataset; red points = sampled subset (9 points). Sampling reduces simulation cost while retaining representativeness.	35

- 3.24 Overlap between RQMC-9 and RQMC-14 sampling sets. The 9-sample configuration is entirely contained within the 14-sample configuration. 38

List of Tables

3.1	Scenario 1 datasets (D2-D7) with varied behavioral conditions.	12
3.2	Scenario 1 summary across Datasets 2-7. Reported are the dataset's evacuation-time variance, the best validation R^2 , the median validation R^2 across all hyperparameter settings, and the configuration(width, activation, #Dense, reg) of the best model.	16
3.3	Best accuracy and typical training time per fit across methods (Scenario 2). Best rows are selected by validation R^2 ; training time is the median over runs.	30
3.4	Comparison of sampling methods in Scenario 2. “# Samples” refers to the number of (E, T, P, RunNo) points used for training. Generalization R^2 is measured on the remainder of the dataset not seen during training.	37

Bibliography

- accu:rate GmbH. (2023). *Crowd:it documentation, version 3.1.0* [Software documentation].
accu:rate GmbH. <https://www.accu-rate.de/doku/en/crowd-it-model>
- Arthur, D., & Vassilvitskii, S. (2006). How slow is the k-means method? *Proceedings of the 22nd Annual Symposium on Computational Geometry*, 144–153.
- Bandini, S., Manzoni, S., & Vizzari, G. (2009). Agent based modeling and simulation: An informatics perspective. *Journal of Artificial Societies and Social Simulation*, 12(4), 4. <http://jasss.soc.surrey.ac.uk/12/4/4.html>
- Bareth, T. (2017). *Baudimensionierung hinsichtlich fluchtwegesicherheit und komfort – eine betrachtung von ingenieurtechnischen berechnungsmethoden vor dem hintergrund der baurechtlichen vorschriften* [Master’s Thesis]. Technische Universität München.
- Bolager, E. L., Burak, I., Datar, C., Sun, Q., & Dietrich, F. (2023). Sampling weights of deep neural networks. *arXiv preprint arXiv:2306.16830*. <https://arxiv.org/abs/2306.16830>
- Cover, T. M., & Thomas, J. A. (1991). *Elements of information theory* (1st). Wiley-Interscience.
- Helbing, D., Farkas, I., & Vicsek, T. (2000). Simulating dynamical features of escape panic. *Nature*, 407(6803), 487–490. <https://doi.org/10.1038/35035023>
- Joblib Development Team. (2023). Joblib: Running python functions as pipeline jobs [Version 1.3.2, accessed September 2025].
- Kneidl, A., & Könnecke, R. (2020). *Fachgutachten zu fluchtwegen in arbeitsstätten – einfluss von wegbreite, treppen, türen und einengungen auf die entfluchtung* (2nd, revised, tech. rep.). Bundesanstalt für Arbeitsschutz und Arbeitsmedizin (BAuA). Dortmund/Berlin/Dresden. <https://doi.org/10.21934/baua:bericht20200116>
- Kuligowski, E. D. (2008). *Modeling human behavior during building fires* (Technical Note No. 1619). National Institute of Standards and Technology (NIST). Gaithersburg, MD. <https://doi.org/10.6028/NIST.TN.1619>
- Kuligowski, E. D., & Peacock, R. D. (2005). *A review of building evacuation models* (Technical Note No. 1471). National Institute of Standards and Technology (NIST). Gaithersburg, MD. <https://doi.org/10.6028/NIST.TN.1471>
- Lloyd, S. (1982). Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28(2), 129–137. <https://doi.org/10.1109/TIT.1982.1056489>

- Moussaïd, M., Helbing, D., & Theraulaz, G. (2011). How simple rules determine pedestrian behavior and crowd disasters. *Proceedings of the National Academy of Sciences*, 108(17), 6884–6888. <https://doi.org/10.1073/pnas.1016507108>
- Owen, A. B. (1998). Scrambling sobol’ and niederreiter–xing points. *Journal of Complexity*, 14(4), 466–489. <https://doi.org/10.1006/jcom.1998.0487>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems (NeurIPS 2019)*.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://scikit-learn.org/stable/>
- Rahma, M., Datar, C., Cukarska, A., & Dietrich, F. (2025). Rapid training of hamiltonian graph networks without gradient descent. *arXiv preprint arXiv:2506.06558*.
- RiMEA e.V. (2016). *Richtlinie für mikroskopische entfluchtungsanalysen (rimea guideline), version 3.0.0* [Published 10 March 2016]. RiMEA e.V. Germany. <https://www.rimea.de>
- Schadschneider, A., Klingsch, W., Klüpfel, H., Kretz, T., Rogsch, C., & Seyfried, A. (2011). Evacuation dynamics: Empirical results, modeling and applications. In S. Albeverio, V. Jentsch, & H. Kantz (Eds.), *Extreme environmental events: Complexity in forecasting and early warning* (pp. 517–550). Springer. https://doi.org/10.1007/978-1-4419-7695-6_30
- Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>
- Sobol’, I. M. (1967). On the distribution of points in a cube and the approximate evaluation of integrals. *USSR Computational Mathematics and Mathematical Physics*, 7(4), 86–112. [https://doi.org/10.1016/0041-5553\(67\)90144-9](https://doi.org/10.1016/0041-5553(67)90144-9)
- Stoyanov, M. (2017). *User manual: Toolkit for adaptive stochastic modeling and non-intrusive approximation (tasmanian)* (tech. rep. No. ORNL/TM-2015/596) (Version 7.0). Oak Ridge National Laboratory. <https://doi.org/10.2172/1369223>
- Zhou, S., Chen, D., Cai, W., Luo, L., Low, M. Y. H., Tian, F., Tay, V. S.-H., Ong, D. W. S., & Hamilton, B. D. (2010). Crowd modeling and simulation technologies. *ACM Transactions on Modeling and Computer Simulation*, 20(4), 20:1–20:35. <https://doi.org/10.1145/1842722.1842725>